



PEMROGRAMAN WEB



www.google.com



PENGGUNA

1. SELECT DATA

- Menampilkan data tabel pengguna, pertama-tama menyiapkan back end terlebih dahulu yaitu berbasis php untuk pengambilan data
- Install axios. Jika sudah ada di package JSON, maka tidak perlu dilakukan install.
- Cara menarik data ada dua jenis yaitu POST dan GET
- POST menyimpan parameter yang di kirim dalam bentuk body, sdangkan GET menyimpan parameter pada url
- Hasil yang dikembalikan yaitu sebuah data berbentuk JSON
- Cara select data pengguna yaitu membuat function dimana pada contoh menggunakan metode GET. Hasil yang diperoleh disimpan pada variabel setter dan getter.

```
1  const Pengguna = () => {
2    const [user, setUser] = useState([]);
3
4    const selectPengguna = async () => {
5      const url = "http://localhost/inventarisweb/penggunaread.php";
6      try {
7        const response = await axios.get(url);
8        console.log(response);
9      } catch (error) {
10       console.log(error);
11     }
12   };
13
14   return (
```

- Lakukan pemanggilan fungsi untuk pertama kali component diload



```
1  useEffect(() => {  
2    selectPengguna();  
3  }, []);  
4  
5  return (
```

- Jika data berhasil di GET maka hasilnya saat di inspect element

```
▼ {data: {...}, status: 200, statusText: 'OK', headers: AxiosHeaders$1, config: {...}, ...} 1  
  ► config: {transitional: {...}, adapter: Array(3), transformRequest: Array  
  ▼ data:  
    ▼ DATA: Array(2)  
      ► 0: {id: 1, username: 'admin1', password: 'adminsatu', nama: 'ADMIN  
      ► 1: {id: 2, username: 'admin2', password: 'admindua', nama: 'ADMIN  
        length: 2  
      ► [[Prototype]]: Array(0)  
      PESAN: ""  
      STATUS: "BERHASIL"  
      ► [[Prototype]]: Object  
      ► headers: AxiosHeaders$1 {content-length: '310', content-type: 'text/h  
      ► request: XMLHttpRequest {onreadystatechange: null, readyState: 4, tim  
        status: 200  
        statusText: "OK"  
      ► [[Prototype]]: Object
```

- Jika sudah berhasil maka console.log pada fungsi select pengguna dihapus diganti dengan penyimpanan ke data user



```
1  try {  
2    const response = await axios.get(url);  
3    setUser(response.data["DATA"]);  
4  } catch (error) {  
5    console.log(error);  
6  }
```

- Pada component pengguna, buat tabel dan lakukan perulangan pada table body

```
1  return (  
2    <>  
3    <Table.Root>  
4      <Table.Header>  
5        <Table.Row>  
6          <Table.ColumnHeader>No</Table.ColumnHeader>  
7          <Table.ColumnHeader>Nama</Table.ColumnHeader>  
8          <Table.ColumnHeader>Username</Table.ColumnHeader>  
9          <Table.ColumnHeader>Password</Table.ColumnHeader>  
10       </Table.Row>  
11     </Table.Header>  
12     <Table.Body>  
13       {user.map((item, index)=> (  
14         <Table.Row key={item.id}>  
15           <Table.Cell>{index + 1}</Table.Cell>  
16           <Table.Cell>{item.nama}</Table.Cell>  
17           <Table.Cell>{item.username}</Table.Cell>  
18           <Table.Cell>{item.password}</Table.Cell>  
19         </Table.Row>  
20       )}  
21     </Table.Body>  
22   </Table.Root>  
23 </>  
24 );  
25 };
```

2. CREATE DATA

Buat sebuah tombol tambah data pada halaman pengguna dan ketika tombol tambah data di klik maka akan pindah ke halaman form tambah data.

- Langkah pertama buat route yang mengarah pada tambah data terlebih dahulu. Route masih di dalam nest route pada dashboard

```
1 <Route path="/" element={<Login />} />
2 <Route path="/dashboard" element={<Dashboard />}>
3   <Route index element={<Home />} />
4   <Route path="pengguna" element={<Pengguna />} />
5   <Route path="pengguna/tambah" element={<PenggunaCreate />} />
6   <Route path="profil" element={<Profil />} />
7 </Route>
8 </Routes>
```

- Buat component baru. Berikan nama `penggunacreate.jsx`

```
1 const PenggunaCreate = () => {
2   return (
3     <>
4       <Box
5         display="flex"
6         flexDirection="column"
7         width="100dvw"
8         height="100dvh"
9         justifyContent="center"
10        alignItems="center"
11      >
12        <Card.Root width="50dvw" shadowColor="bg.emphasized" shadow="lg">
13          <CardHeader>
14            <CardTitle>
15              <Text>Form Tambah Pengguna</Text>
16            </CardTitle>
17          </CardHeader>
18        </Card.Root>
19      </Box>
20    </>
21  );
22 };
23
24 export default PenggunaCreate;
```

- Pada halaman pengguna, buat tombol sebagai link untuk pindah halaman ke form tambah

```
1  return (  
2    <>  
3    <Heading size="xl" textAlign="center" padding="10px">  
4      Tabel Pengguna  
5    </Heading>  
6    <Box padding="10px">  
7      <Button as={Link} to="tambah" variant="solid" bgColor="teal">  
8        Tambah Pengguna  
9      </Button>  
10   </Box>  
11   <Table.Root>  
12     <Table.Header>
```

Jika sudah berhasil pindah halaman. Tahap berikutnya yaitu input dan kirim data pada form tambah. Sistem mirip dengan form login

Form Tambah Pengguna



```

1  const PenggunaCreate = () => {
2      return (
3          <>
4              <Box
5                  display="flex"
6                  flexDirection="column"
7                  width="100dvw"
8                  height="100dvh"
9                  justifyContent="center"
10                 alignItems="center"
11             >
12                 <Card.Root width="50dvw" shadowColor="bg.emphasized" shadow="lg">
13                     <CardHeader>
14                         <CardTitle>
15                             <Text>Form Tambah Pengguna</Text>
16                         </CardTitle>
17                     </CardHeader>
18                     <CardBody gapY="10px">
19                         <Input placeholder="Username" type="text" />
20                         <Input placeholder="Password" type="password" />
21                         <Input placeholder="Nama" type="text" />
22                         <Button backgroundColor="teal" color="white" borderRadius="10px">
23                             <Text>Tambah Pengguna</Text>
24                         </Button>
25                         <Button variant="outline" borderRadius="10px">
26                             <Text>Kembali</Text>
27                         </Button>
28                     </CardBody>
29                 </Card.Root>
30             </Box>
31         </>
32     );
33 };

```

- Pada penggunacreate, buat setter getter untuk setiap input dan usenavigate untuk pindah halaman



```

1  const PenggunaCreate = () => {
2      const [username, setUsername] = useState("");
3      const [password, setPassword] = useState("");
4      const [nama, setNama] = useState("");
5      const navigate = useNavigate();
6
7      return (

```

- Buat fungsi handleSimpan yang diarahkan ke php yang dituju. Hasilnya disimpan pada response dan akan ditampilkan dalam console.log

```
1  const handleTambah = async () => {
2    const url = "http://localhost/inventarisweb/penggunainsert.php";
3    const body = { username: username, password: password, nama: nama };
4
5    try {
6      const response = await axios.post(url, body);
7      console.log(response);
8    } catch (error) {
9      console.log(error);
10   }
11 };
12
13 return (
```

- Buat onClick ketika tombol simpan diklik

```
1  <Button onClick={()=>{handleTambah()}} backgroundColor="teal" color="white" borderRadius="10px">
2    <Text>Tambah Pengguna</Text>
3  </Button>
```

- Pada tombol kembali, lakukan pindah halaman ke menu dashboard/pengguna

```
1  <Button as={Link} to="/dashboard/pengguna" variant="outline" borderRadius="10px">
2    <Text>Kembali</Text>
3  </Button>
```


- Tidak lupa untuk menambahkan onChange untuk setter setiap variable

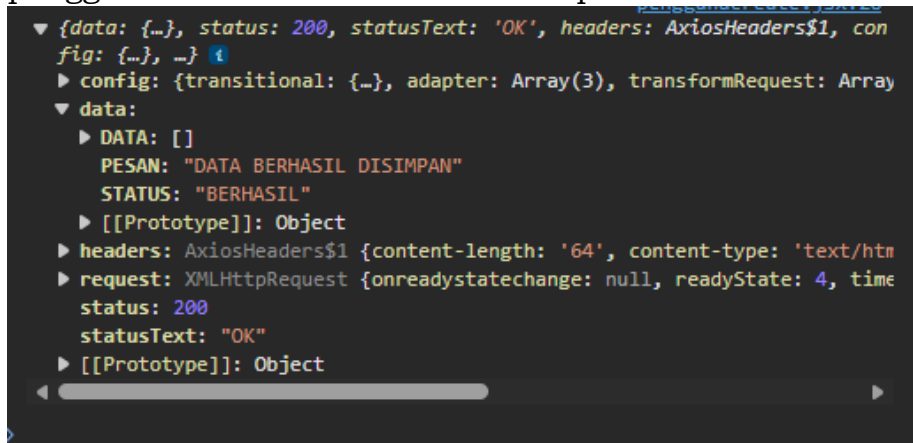


```

1  <Input
2    onChange={(e) => {
3      setUsername(e.target.value);
4    }}
5    placeholder="Username"
6    type="text"
7  />
8  <Input
9    onChange={(e) => {
10     setPassword(e.target.value);
11   }}
12   placeholder="Password"
13   type="password"
14 />
15 <Input
16   onChange={(e) => {
17     setNama(e.target.value);
18   }}
19   placeholder="Nama"
20   type="text"
21 />

```

- Silahkan coba lakukan penambahan data, jika tombol tambah pengguna telah di klik maka cek inspect element.



```

▼ {data: {...}, status: 200, statusText: 'OK', headers: AxiosHeaders$1, config: {...}, ...}
  ► config: {transitional: {...}, adapter: Array(3), transformRequest: Array
  ▼ data:
    ► DATA: []
    PESAN: "DATA BERHASIL DISIMPAN"
    STATUS: "BERHASIL"
    ► [[Prototype]]: Object
  ► headers: AxiosHeaders$1 {content-length: '64', content-type: 'text/html'
  ► request: XMLHttpRequest {onreadystatechange: null, readyState: 4, time
    status: 200
    statusText: "OK"
  ► [[Prototype]]: Object

```

- Jika data tampil seperti ini artinya data berhasil ditambah, Langkah selanjutnya adalah console.log hasil pada handleTambah diganti ke navigasi. Jika berhasil maka pindah ke halaman pengguna.



```
1  const handleTambah = async () => {
2    const url = "http://localhost/inventarisweb/penggunainsert.php";
3    const body = { username: username, password: password, nama: nama };
4
5    try {
6      const response = await axios.post(url, body);
7      if (response.data.STATUS === "BERHASIL") {
8        navigate("/dashboard/pengguna");
9      } else {
10       navigate("/dashboard/pengguna/tambah");
11     }
12   } catch (error) {
13     console.log(error);
14   }
15   };
```

3. UPDATE DATA

Kembali pada halaman pengguna. Buat setiap column memiliki aksi untuk update

☰

My Inventory

🗖

Tabel Pengguna

Tambah Pengguna

No	Nama	Username	Password	Aksi
1	ADMIN 1	admin1	adminsatu	UbahHapus
2	ADMIN 2	admin2	admindua	UbahHapus
3	test	test	test	UbahHapus
4	test	test	test	UbahHapus
5	asa	asas	asas	UbahHapus

Form Update Pengguna

admin2

.....

ADMIN 2

Update Pengguna

Kembali

Yang berbeda antara form update dan tambah yaitu ketika update terjadi oper parameter antar halaman. Ini bisa dilakukan oper parameter lewat route

- Langkah pertama yaitu buat halaman form update. Berikan nama penggunaupdate.jsx. Form sama persis modelnya seperti form tambah data.



```
1  const PenggunaUpdate = () => {
2    const [username, setUsername] = useState("");
3    const [password, setPassword] = useState("");
4    const [nama, setNama] = useState("");
5    const navigate = useNavigate();
6
7    const handleUpdate={()=>{
8
9    }
10
11    return (
12      <>
13        <Box
14          display="flex"
15          flexDirection="column"
16          width="100dvw"
17          height="100dvh"
18          justifyContent="center"
19          alignItems="center"
20        >
21          <Card.Root width="50dvw" shadowColor="bg.emphasized" shadow="1g">
22            <CardHeader>
23              <CardTitle>
24                <Text>Form Update Pengguna</Text>
25              </CardTitle>
26            </CardHeader>
27            <CardBody gapY="10px">
28              <Input
29                onChange={(e) => {
30                  setUsername(e.target.value);
31                }}
32                placeholder="Username"
33                type="text"
34              />
35              <Input
36                onChange={(e) => {
37                  setPassword(e.target.value);
38                }}
39                placeholder="Password"
40                type="password"
41              />
42              <Input
43                onChange={(e) => {
44                  setNama(e.target.value);
45                }}
46                placeholder="Nama"
47                type="text"
48              />
49              <Button
50                onClick={() => {}}
51                backgroundColor="teal"
52                color="white"
53                borderRadius="10px"
54              >
55                <Text>Update Pengguna</Text>
56              </Button>
57              <Button
58                as={Link}
59                to="/dashboard/pengguna"
60                variant="outline"
61                borderRadius="10px"
62              >
63                <Text>Kembali</Text>
64              </Button>
65            </CardBody>
66          </Card.Root>
67        </Box>
68      </>
69    );
70  };
```

- Buat route untuk halaman update dimana terdapat route bisa oper parameter



```

1  <BrowserRouter>
2    <Routes>
3      <Route path="/" element={<Login />} />
4      <Route path="/dashboard" element={<Dashboard />} />
5      <Route index element={<Home />} />
6      <Route path="pengguna" element={<Pengguna />} />
7      <Route path="pengguna/tambah" element={<PenggunaCreate />} />
8      <Route path="pengguna/update/:id" element={<PenggunaUpdate />} />
9      <Route path="profil" element={<Profil />} />
10   </Route>
11 </Routes>
12 </BrowserRouter>

```

- Pada halaman pengguna, pada tombol update lakukan pindah halaman disertakan dengan parameter yang dioper



```

1  <Table.Cell>{index + 1}</Table.Cell>
2  <Table.Cell>{item.nama}</Table.Cell>
3  <Table.Cell>{item.username}</Table.Cell>
4  <Table.Cell>{item.password}</Table.Cell>
5  <Table.Cell>
6    <Button
7      as={Link}
8      to={`update/${item.id}`}
9      margin="2px"
10     bgColor="blue.400"
11   >
12     Ubah
13 </Button>

```

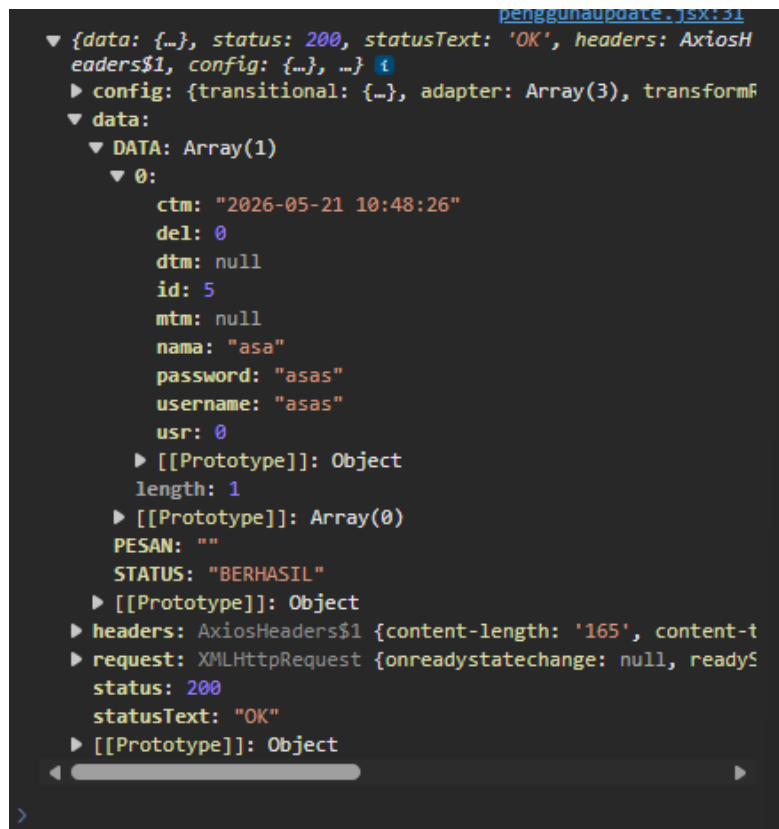
- Cara menangkap parameter pada form update bisa menggunakan useparams. Silahkan console log untuk cek berhasil oper atau tidak

```
1  const PenggunaUpdate = () => {
2    const [username, setUsername] = useState("");
3    const [password, setPassword] = useState("");
4    const [nama, setNama] = useState("");
5    const navigate = useNavigate();
6
7    const { id } = useParams();
8    console.log(id);
```

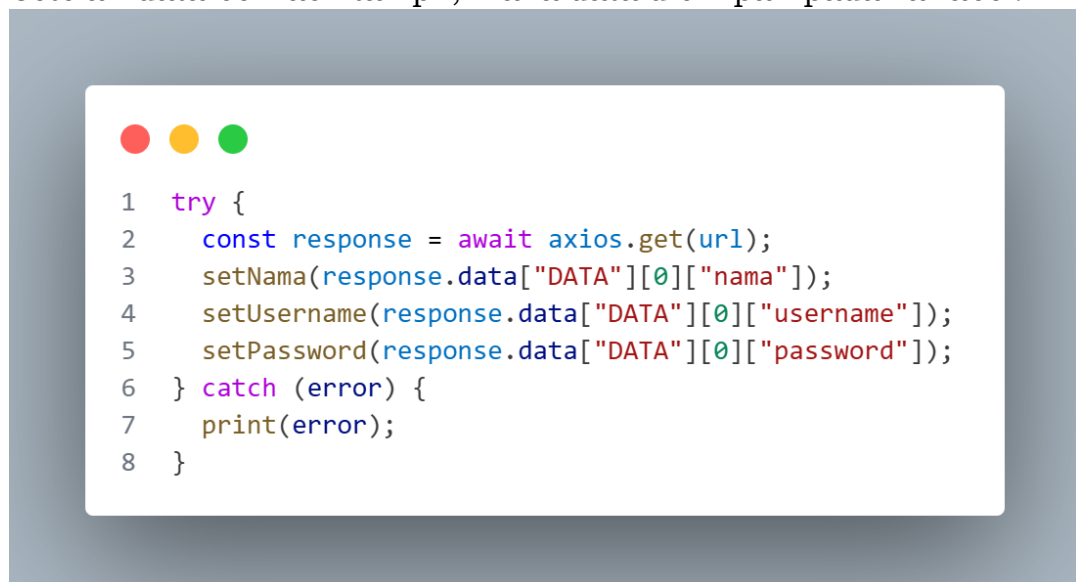
```
5                                     penggunaupdate.jsx:24
5                                     penggunaupdate.jsx:24
```

- Karena update berarti mengambil data pengguna tertentu. Maka lakukan select data pada php yang dituju. Tampilkan hasil response pada console.log.

```
1
2  const selectSatuPengguna = async () => {
3    const url = `http://localhost/inventarisweb/satupengguna/read.php?id=${id}`;
4
5    try {
6      const response = await axios.get(url);
7      console.log(response);
8    } catch (error) {
9      print(error);
10   }
11 };
12
13 useEffect(() => {
14   selectSatuPengguna();
15 }, []);
16
17 return (
```



- Setelah data berhasil tampil, maka data disimpan pada variabel.



- Setelah sudah simpan pada variabel maka bisa tampilkan pada form



```
1  <Input
2    onChange={(e) => {
3      setUsername(e.target.value);
4    }}
5    placeholder="Username"
6    type="text"
7    value={username}
8  />
9  <Input
10   onChange={(e) => {
11     setPassword(e.target.value);
12   }}
13   placeholder="Password"
14   type="password"
15   value={password}
16 />
17 <Input
18   onChange={(e) => {
19     setName(e.target.value);
20   }}
21   placeholder="Nama"
22   type="text"
23   value={nama}
24 />
```

- Langkah selanjutnya membuat tombol ubah berfungsi untuk update data. Buat fungsi terlebih dahulu



```

1  const handleUpdate = async () => {
2    const url = "http://localhost/inventarisweb/penggunaupdate.php";
3    const body = { username: username, password: password, nama: nama, id: id };
4
5    try {
6      const response = await axios.post(url, body);
7      console.log(response);
8    } catch (error) {
9      print(error);
10   }
11 };
12
13 return (

```

- Pada tombol edit, tambahkan onClick

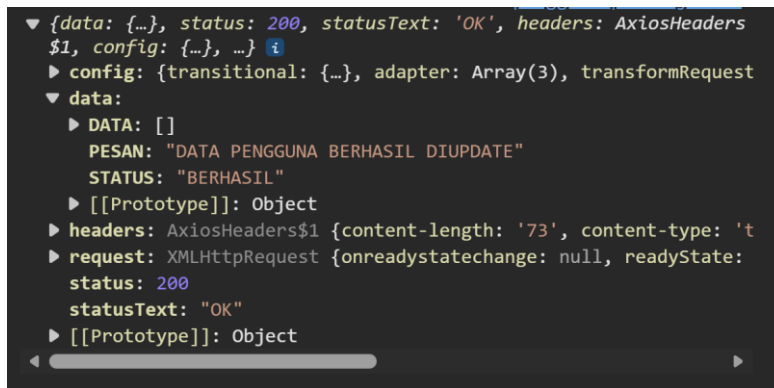


```

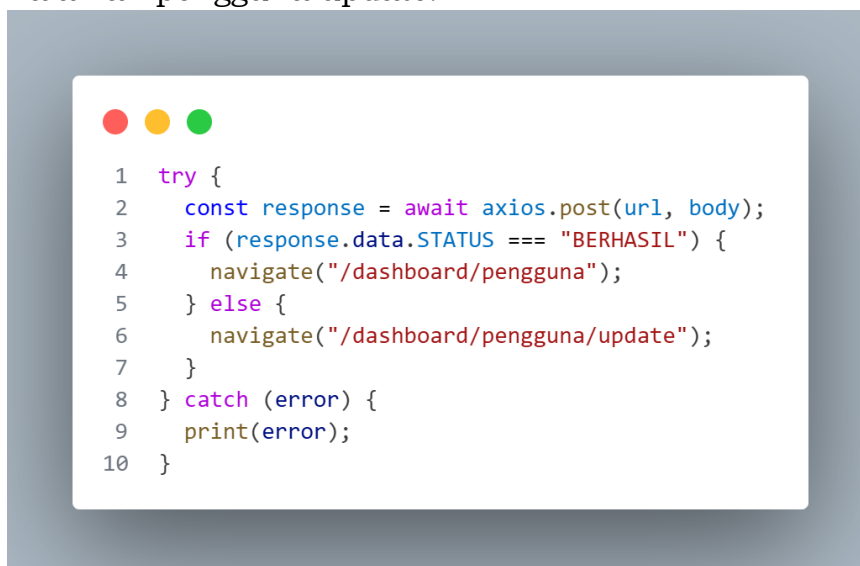
1  <Button
2    onClick={() => {
3      handleUpdate();
4    }}
5    backgroundColor="teal"
6    color="white"
7    borderRadius="10px"
8  >
9    <Text>Update Pengguna</Text>
10 </Button>

```

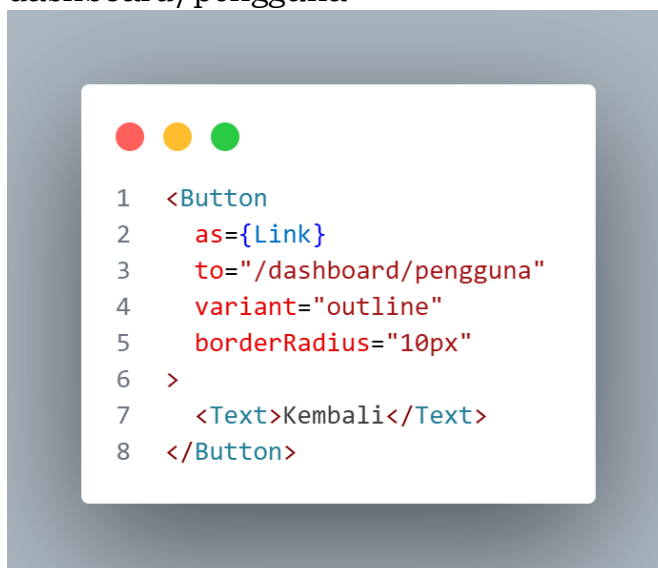
- Silahkan test tombol update, jika data muncul berhasil pada console.log berarti update pengguna berhasil



- Jika sudah berhasil maka buat kondisi jika data berhasil di update maka navigasi ke halaman pengguna, jika gagal maka tetap di halaman pengguna update.

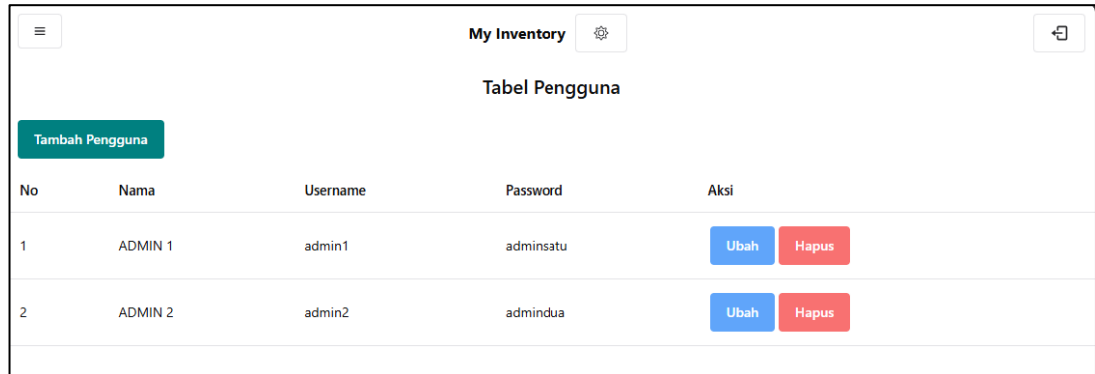


- Pada tombol kembali berfungsi untuk kembali ke halaman dashboard/pengguna

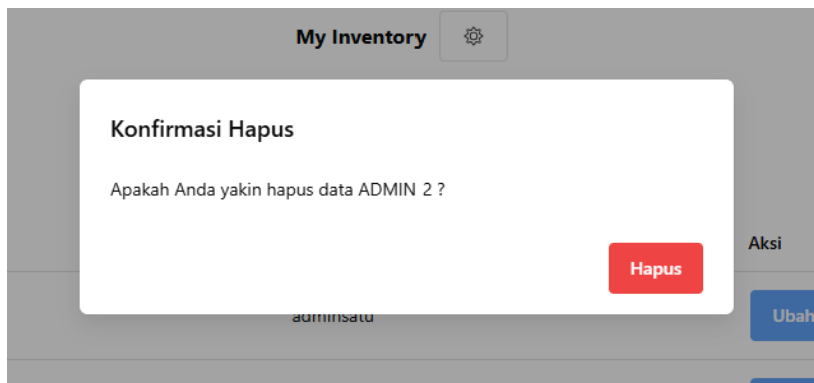


4. DELETE DATA

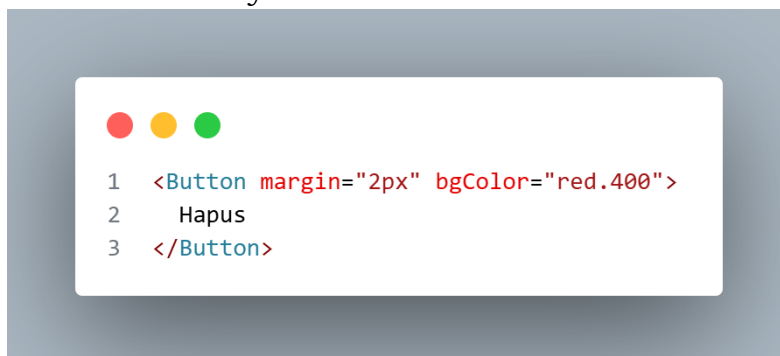
Pada delete data, posisi ada di dekat tombol update, ketika diklik maka akan muncul dialog alert.



No	Nama	Username	Password	Aksi
1	ADMIN 1	admin1	adminsatu	<button>Ubah</button> <button>Hapus</button>
2	ADMIN 2	admin2	admindua	<button>Ubah</button> <button>Hapus</button>



- Langkah pertama adalah tambahkan code dialog sebagai tombol delete
- Code sebelumnya



- Ganti button hapus menjadi dialog. Code dialog bisa didapatkan templatanya pada doc ChakraUI v3.



```
1 <Dialog.Root>
2   <Dialog.Trigger asChild>
3     <Button margin="2px" bgColor="red.400">
4       Hapus
5     </Button>
6   </Dialog.Trigger>
7   <Portal>
8     <Dialog.Backdrop />
9     <Dialog.Positioner>
10      <Dialog.Content>
11        <Dialog.Header>
12          <Dialog.Title>Dialog Title</Dialog.Title>
13        </Dialog.Header>
14        <Dialog.Body>
15          <p>
16            Lorem ipsum dolor sit amet, consectetur adipiscing
17            elit. Sed do eiusmod tempor incididunt ut labore et
18            dolore magna aliqua.
19          </p>
20        </Dialog.Body>
21        <Dialog.Footer>
22          <Dialog.ActionTrigger asChild>
23            <Button variant="outline">Cancel</Button>
24          </Dialog.ActionTrigger>
25          <Button>Save</Button>
26        </Dialog.Footer>
27        <Dialog.CloseTrigger asChild>
28          <CloseButton size="sm" />
29        </Dialog.CloseTrigger>
30      </Dialog.Content>
31    </Dialog.Positioner>
32  </Portal>
33 </Dialog.Root>
```

- Rapikan content di dalam dialog



```
1 <Dialog.Root>
2   <Dialog.Trigger asChild>
3     <Button margin="2px" bgColor="red.400">
4       Hapus
5     </Button>
6   </Dialog.Trigger>
7   <Portal>
8     <Dialog.Backdrop />
9     <Dialog.Positioner>
10    <Dialog.Content>
11      <Dialog.Header>
12        <Dialog.Title>Konfirmasi Hapus</Dialog.Title>
13      </Dialog.Header>
14      <Dialog.Body>
15        <Text>
16          Apakah Anda yakin hapus data {item.nama} ?
17        </Text>
18      </Dialog.Body>
19      <Dialog.Footer>
20        <Dialog.ActionTrigger asChild>
21          <Button bgColor="red.500">Hapus</Button>
22        </Dialog.ActionTrigger>
23      </Dialog.Footer>
24    </Dialog.Content>
25  </Dialog.Positioner>
26 </Portal>
27 </Dialog.Root>
```

- Tambahkan OnClick pada button hapus di dalam dialog



```
1 <Dialog.Footer>
2   <Dialog.ActionTrigger asChild>
3     <Button variant="outline">Batal</Button>
4   </Dialog.ActionTrigger>
5   <Button
6     bgColor="red.500"
7     onClick={() => {
8       handleHapus(item.id);
9     }}
10  >
11    Hapus
12  </Button>
13 </Dialog.Footer>
```

- Buat fungsi untuk handlehapus

```
1  const handleHapus = async (id) => {  
2    const url = "http://localhost/inventarisweb/penggunadelete.php";  
3    const body = { id: id };  
4  
5    try {  
6      const response = await axios.post(url, body);  
7  
8      if (response.data.STATUS === "BERHASIL") {  
9        await selectPengguna();  
10     }  
11   } catch (error) {  
12     console.log(error);  
13   }  
14   };
```

SELAMAT ANDA SUDAH BERHASIL
MEMBUAT WEB BERBASIS REACT JS, CHAKRA UI V3 dan PHP